

PART 1 - FUNCTIONS

- Q1. A function is a reusable block of code that performs a specific task in program. The welcomeMessage function was created to print welcome message for the school system, helping reduce repeated code and improving program organization and readability.
- Q2. Named parameters allows arguments to be passed using their parameter names, which improves code clarity and readability. They reduce mistakes when many parameters exist and allow arguments to be passed in any order, making functions easier to use and understand.
- Q3. Optional parameters allow a functions to be called with or without certain arguments. It is function subject is optional, so if it is not provided, the function prints "subject not assigned" making the function flexible and safe for missing values.

PART 2: CONSTRUCTORS & CLASSES.

- Q4. A constructor is a special method that runs automatically when an object is created. It is used to initialize class variables such as name and age in the student class, ensuring every object starts with proper and meaningful data values.
- Q5. An object is an instance of a class created using a constructor. It represents a real-world entity, and its properties can be accessed using the dot operator. Objects allow us to use class variables and functions in the program.

PART 3: INHERITANCE.

- Q6. A class is a blueprint or template used to create objects. It defines properties and behaviors for objects. The Person class contains a name variable and an introduce function, allowing multiple objects to share the same structure and behavior.
- Q7. Inheritance allows one class to reuse properties and methods from another class. The student class inherits from Person, so it can use the name variable and introduce function without rewriting code, reducing duplication and improving program maintainability.

PART 4: INTERFACES

- Q8. An Interface defines a set of methods that a class must implement. In Part, abstract classes act as interfaces. The Registrable interface declares the register course method, ensuring that any implementing class follows a specific structure and behavior.
- Q9. Implementing an interface means a class must provide concrete implementations for all declared methods. The student class implements Registrable, so it must define register course, ensuring consistency and enforcing rules for all student objects in the program.

PART 5: MIXINS

- Q10. A Mixin is a way to reuse code across multiple class without inheritance. AttendanceMixin adds attendance tracking behavior to a class, storing an attendance counter and providing a function to increase it, promoting code reuse and modular programming.
- Q11. Mixins are applied using the with key word to add behavior to a class. By applying AttendanceMixin to student, the class gains attendance tracking features without changing its core structure, making the program more flexible and reusable.

PART 6: COLLECTIONS

- Q12. A List is a collection that stores multiple items in a specific order and allows access using indexes. It is useful for storing multiple student objects and enables easy addition, removal, and iteration through students in the program.
- Q13. A map stores data in key-value pairs, where each key uniquely identifies a value. It is useful for fast data access, such as retrieving a student using an ID and printing student names efficiently without searching through the entire collection.

PART 7: ANONYMOUS AND LAMBDA FUNCTION

- Q14. An anonymous function is a function without a name that is used directly where needed. It is useful for short tasks like printing student names from a list, improving code readability and keeping logic close to where it is used.
- Q15. Arrow functions provide a short and concise way to write functions with a single expression. They reduce code size and improve readability, making them ideal for simple tasks such as printing greeting message or performing quick operations.

PART 8: ASYNCHRONOUS PROGRAMMING.

- Q16. Async functions allow a program to perform long tasks like loading data without stopping the entire program. The `wait` keyword pauses execution until the task finishes while allowing other operations to continue, making applications more efficient and responsive.
- Q17. Asynchronous programming helps real applications stay responsive by preventing the user interface from freezing during long tasks like network requests or file reading. It ensures smooth user experiences and is widely used in modern mobile and web applications.

PART 9: INTEGRATION CHALLENGE.

- Q18. Mixins are useful because they allow a class to reuse functionality from multiple sources without a strict parent-child relationship. Inheritance allows only one parent class, while mixins add specific behaviors, making the code more flexible and modular.
- Q19. NotificationMixin was created to add notification behavior when a student is registered for a course. By applying the mixin to Student, the class gains new functionality without modifying its main structure, improving modularity and reusability.
- Q20. Learning Dart helps in understanding Flutter because Flutter uses Dart as its main programming language. Dart concepts such as classes, functions, async programming, and object-oriented principles are used in Flutter, making app development easier and more structured.